The Processor: Part I

CSE 4305: Computer Organization and Architecture

Mohammad Ishrak Abedin
Lecturer, Department of CSE

29 December 2025

Islamic University of Technology

Table of Contents

Introduction
Logic Design Convention
Building a Datapath
A Simple Implementation
An Overview of Pipelining

In a major matter, no details are small.

French Proverb

Introduction

Introduction

- Review: CPU performance factors:
  - Instruction count — determined by ISA and compiler
  - CPI and Cycle time — determined by CPU hardware

Introduction

- Review: CPU performance factors:
  - Instruction count — determined by ISA and compiler
  - CPI and Cycle time — determined by CPU hardware
- Plan: We will examine two RISC-V implementations
  - A simplified version for understanding
  - A more realistic pipelined version

Introduction

- Review: CPU performance factors:
  - Instruction count — determined by ISA and compiler
  - CPI and Cycle time — determined by CPU hardware
- Plan: We will examine two RISC-V implementations
  - A simplified version for understanding
  - A more realistic pipelined version
- Coverage: Simple subset of instructions, shows most aspects
  - Memory reference: `lw`, `sw`
  - Arithmetic/logical: `add`, `sub`, `and`, `or`
  - Control transfer: `beq`

High-Level Instruction Execution Overview

- $PC \rightarrow$ instruction memory, fetch instruction

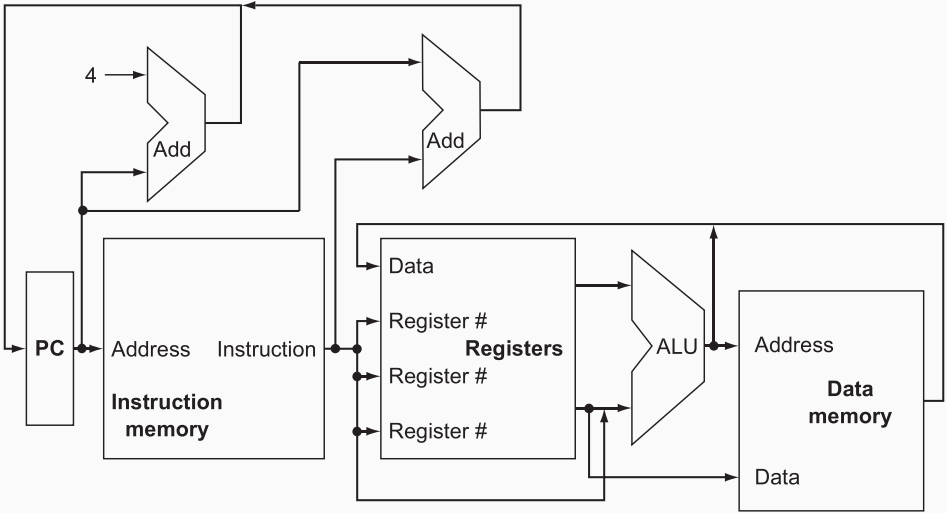
High-Level Instruction Execution Overview

- $PC \rightarrow$ instruction memory, fetch instruction
- Register numbers $\rightarrow$ register file, read registers

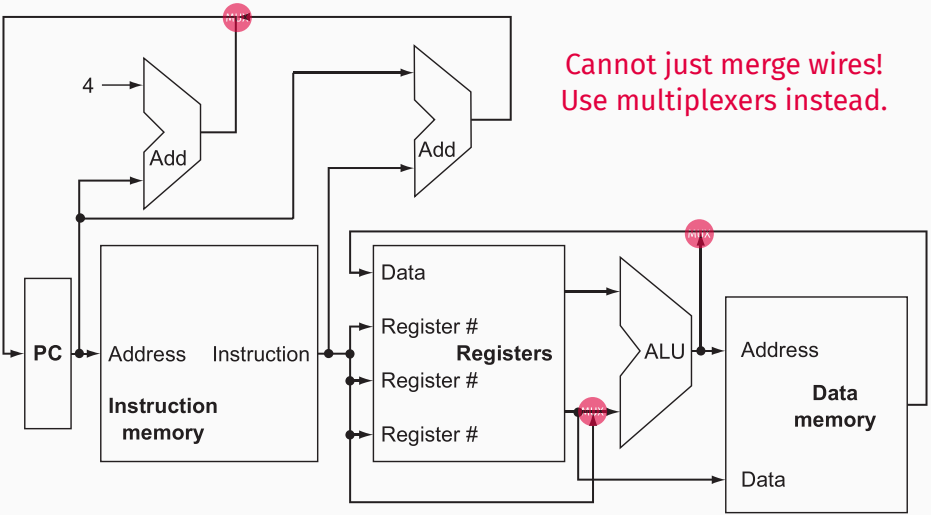
High-Level Instruction Execution Overview

- $PC \rightarrow$ instruction memory, fetch instruction
- Register numbers $\rightarrow$ register file, read registers
- Depending on instruction class:
  - Use ALU to calculate:
    - Arithmetic result
    - Memory address for load/store
    - Branch comparison
  - Access data memory for load/store
  - $PC \leftarrow$ target address of branch or $PC + 4$

CPU Overview

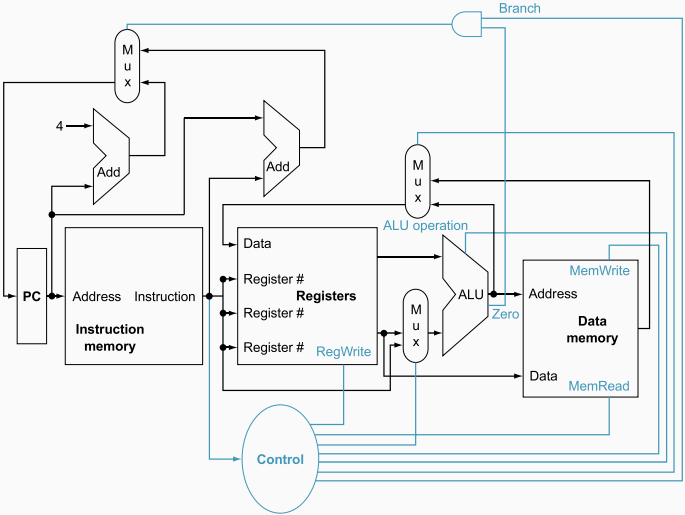


CPU Overview with Multiplexer



Cannot just merge wires!
Use multiplexers instead.

CPU Overview with Mux and Control



Designing Logic

Logic Design Basics

- Information encoded in binary
  - Low voltage = 0, High voltage = 1
  - One wire per bit
  - Multi-bit data encoded on multi-wire bus

Logic Design Basics

- Information encoded in binary
  - Low voltage = 0, High voltage = 1
  - One wire per bit
  - Multi-bit data encoded on multi-wire bus
- Two different types of logic elements

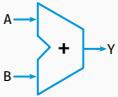
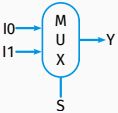
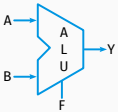
Logic Design Basics

- Information encoded in binary
  - Low voltage = 0, High voltage = 1
  - One wire per bit
  - Multi-bit data encoded on multi-wire bus
- Two different types of logic elements
  - Combinational element
    - Operate on data
    - Output is a function of input

Logic Design Basics

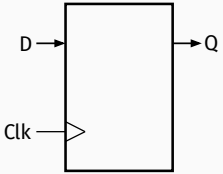
- Information encoded in binary
  - Low voltage = 0, High voltage = 1
  - One wire per bit
  - Multi-bit data encoded on multi-wire bus
- Two different types of logic elements
  - Combinational element
    - Operate on data
    - Output is a function of input
  - State (sequential) element
    - Store information

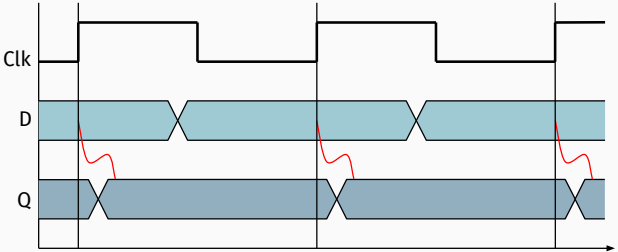
Combination Elements

Name	Function	Diagram
AND-gate	$Y = A\&B$	
Adder	$Y = A + B$	
Multiplexer	$Y = S?I1 : I0$	
Arithmetic/Logic Unit	$Y = F(A, B)$	

Sequential Elements

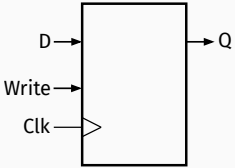
- Register: stores data in a circuit
  - Uses a clock signal to determine when to update the stored value
  - Edge-triggered: update when `Clk` changes from 0 to 1 (rising edge)

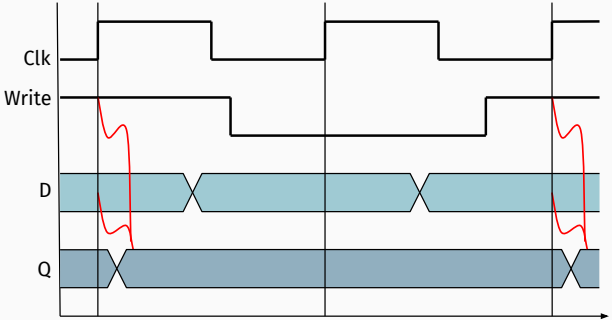




Sequential Elements with Write Signal

- Register with write control
  - Only updates on clock edge when write control input is 1
  - Used when stored value is required later



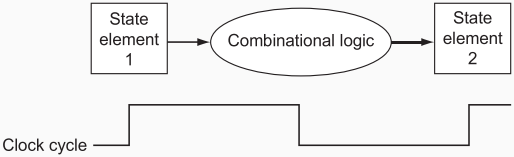


Clocking Methodology

- Combinational logic transforms data during clock cycles
  - Between clock edges
  - Input from state elements, output to state element
  - Longest delay determines clock period

Clocking Methodology

- Combinational logic transforms data during clock cycles
  - Between clock edges
  - Input from state elements, output to state element
  - Longest delay determines clock period



Building a Datapath

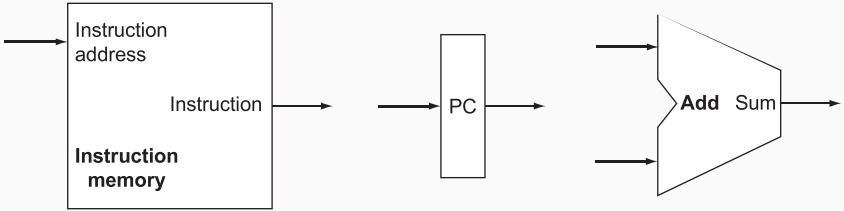
Building a Datapath

- Datapath: Elements that process data and addresses in the CPU
  - Registers, ALUs, multiplexers, memories, etc.

Building a Datapath

- Datapath: Elements that process data and addresses in the CPU
  - Registers, ALUs, multiplexers, memories, etc.
- We will build a simple RISC-V datapath incrementally, refining the overview design

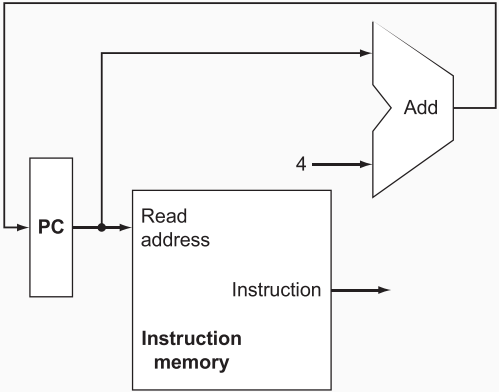
Instruction Fetch Elements



a. Instruction memory b. Program counter c. Adder

Instruction Fetch Datapath

Increment by 4 for the next instruction



Arithmetic/Logic or R-Format Instructions

- Read two register operands

Arithmetic/Logic or R-Format Instructions

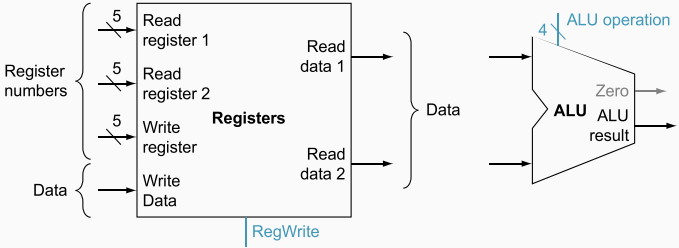
- Read two register operands
- Perform arithmetic/logic operation

Arithmetic/Logic or R-Format Instructions

- Read two register operands
- Perform arithmetic/logic operation
- Write register result

Arithmetic/Logic or R-Format Instructions

- Read two register operands
- Perform arithmetic/logic operation
- Write register result



a. Registers b. ALU

Load/Store Instructions

- Read register operands

Load/Store Instructions

- Read register operands
- Calculate address using 12-bit offset
  - Use ALU, but need to sign-extend offset first
  - Interpretation of the offset is instruction format dependent

Load/Store Instructions

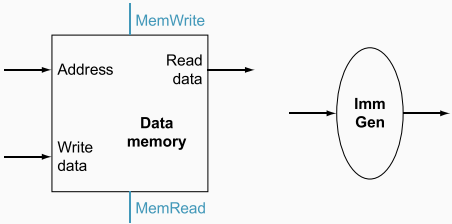
- Read register operands
- Calculate address using 12-bit offset
  - Use ALU, but need to sign-extend offset first
  - Interpretation of the offset is instruction format dependent
- Load: Read memory and update register

Load/Store Instructions

- Read register operands
- Calculate address using 12-bit offset
  - Use ALU, but need to sign-extend offset first
  - Interpretation of the offset is instruction format dependent
- Load: Read memory and update register
- Store: Write register value to memory

Load/Store Instructions

- Read register operands
- Calculate address using 12-bit offset
  - Use ALU, but need to sign-extend offset first
  - Interpretation of the offset is instruction format dependent
- Load: Read memory and update register
- Store: Write register value to memory



a. Data memory unit b. Immediate generation unit

Branch Instruction

- Read register operands

Branch Instruction

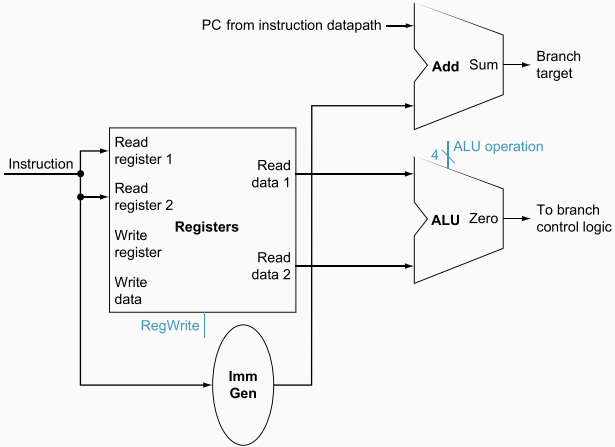
- Read register operands
- Compare operands: use ALU, subtract, and check Zero output

Branch Instruction

- Read register operands
- Compare operands: use ALU, subtract, and check Zero output
- Calculate target address
  - Sign-extend displacement
  - Shift left 1 place (halfword displacement)
  - Add to the *PC* value

Branch Instruction Datapath

Mainly reroute wires within existing components



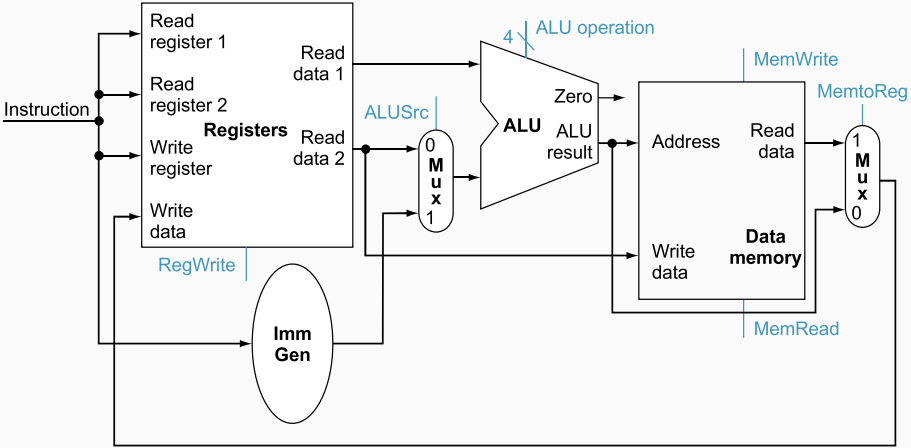
Composing the Elements

- First-cut datapath does an instruction in one clock cycle
  - Each datapath element can only do one function at a time
  - Hence, separate instruction and data memories are required

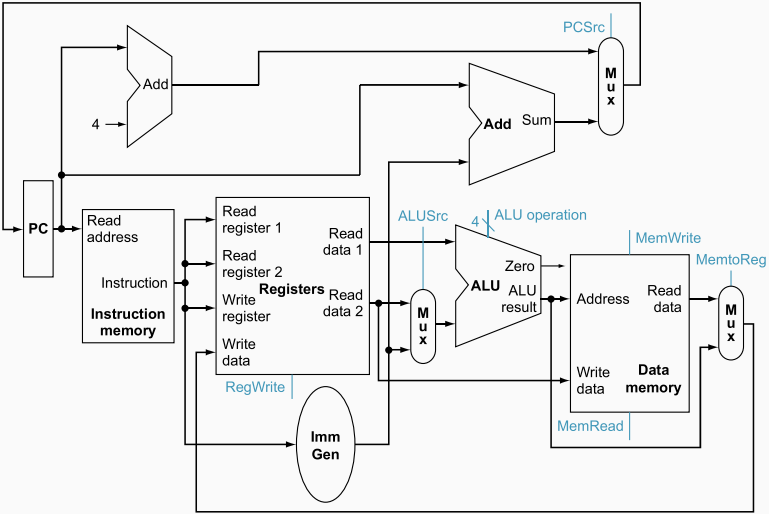
Composing the Elements

- First-cut datapath does an instruction in one clock cycle
  - Each datapath element can only do one function at a time
  - Hence, separate instruction and data memories are required
- Use multiplexers where alternate data sources are used for different instructions

R-Type and Memory Instructions Datapath



Full Simple Datapath for the Core RISC-V Architecture



Simple Implementation

ALU Control

The ALU is used for:

- Load/Store: Function $\rightarrow$ Add

ALU Control

The ALU is used for:

- Load/Store: Function $\rightarrow$ Add
- Branch: Function $\rightarrow$ Subtract

ALU Control

The ALU is used for:

- Load/Store: Function $\rightarrow$ Add
- Branch: Function $\rightarrow$ Subtract
- R-Type: Function $\rightarrow$ Depends on the opcode

ALU Control

The ALU is used for:

- Load/Store: Function → Add
- Branch: Function → Subtract
- R-Type: Function → Depends on the opcode

ALU Control Lines	Function
0000	AND
0001	OR
0010	Add
0110	Subtract

ALU Control with ALUOp

Assume 2-bit ALUOp derived from opcode

- Combinational logic derives ALU control

ALU Control with ALUOp

Assume 2-bit ALUOp derived from opcode

- Combinational logic derives ALU control

Instruction Opcode	ALUOp	Operation	Funct7 Field	Funct3 Field	Desired ALU Action	ALU Control Input
`lw`	00	load word	XXXXXXX	XXX	add	0010
`sw`	00	store word	XXXXXXX	XXX	add	0010
`beq`	01	branch if equal	XXXXXXX	XXX	subtract	0110
R-type	10	add	0000000	000	add	0010
R-type	10	sub	0100000	000	subtract	0110
R-type	10	and	0000000	111	AND	0000
R-type	10	or	0000000	110	OR	0001

The Main Control Unit

Control signals are derived from instructions

The Main Control Unit

Control signals are derived from instructions

Name/ Bit Pos.	Fields 31:25	24:20	19:15	14:12	11:7	6:0
R-type	funct7	rs2	rs1	funct3	rd	opcode
I-type	immediate[11:0]		rs1	funct3	rd	opcode
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

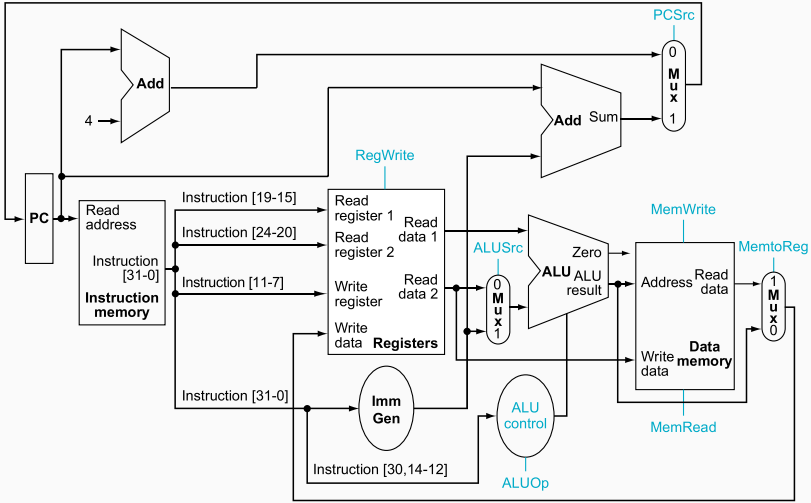
The Main Control Unit

Control signals are derived from instructions

Name/ Bit Pos.	Fields 31:25	24:20	19:15	14:12	11:7	6:0
R-type	funct7	rs2	rs1	funct3	rd	opcode
I-type	immediate[11:0]		rs1	funct3	rd	opcode
S-type	immed[11:5]	rs2	rs1	funct3	immed[4:0]	opcode
SB-type	immed[12,10:5]	rs2	rs1	funct3	immed[4:1,11]	opcode

ALUOp		Funct7 Field							Funct3 Field			
ALUOp1	ALUOp0	I[31]	I[30]	I[29]	I[28]	I[27]	I[26]	I[25]	I[14]	I[13]	I[12]	Operation
0	0	X	X	X	X	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	X	X	X	X	0110
1	X	0	0	0	0	0	0	0	0	0	0	0010
1	X	0	1	0	0	0	0	0	0	0	0	0110
1	X	0	0	0	0	0	0	0	1	1	1	0000
1	X	0	0	0	0	0	0	0	1	1	0	0001

Datapath with Multiplexors and Controls Identified



Effect of Control Signals

Signal Name	Effect when Deasserted	Effect when Asserted
RegWrite	None	The register on the Write register input is written with the value on the Write data input
ALUSrc	The second ALU operand comes from the second register file output (Read data 2)	The second ALU operand is the sign-extended 12 bits of the instruction
PCSrc	The *PC* is replaced by the output of the adder that computers the value of $PC + 4$	The *PC* is replaced by the output of the adder that computes the branch target
MemRead	None	Data memory contents designated by the address input are put on the Read data output
MemWrite	None	Data memory contents designated by the address input are replaced by the value on the Write data input
MemtoReg	The value fed to the register Write data input comes from the ALU	The value fed to the register Write data input comes from the data memory

add, slt, add, addi, lw, jalr (any instructions that save a value in a reg)

I type, sw, lw

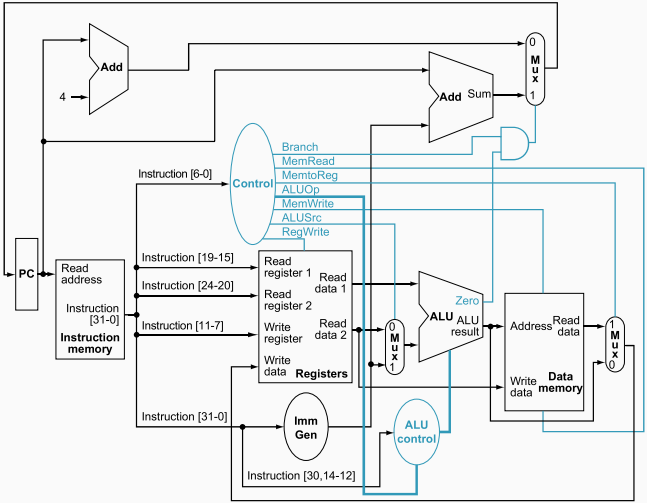
branch and jal

lw

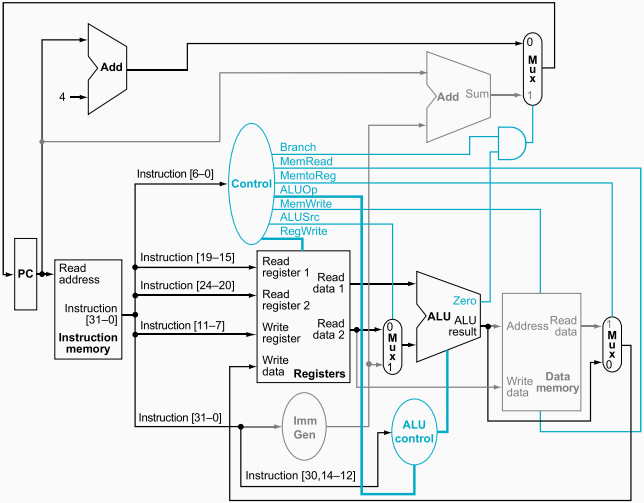
sw

loads

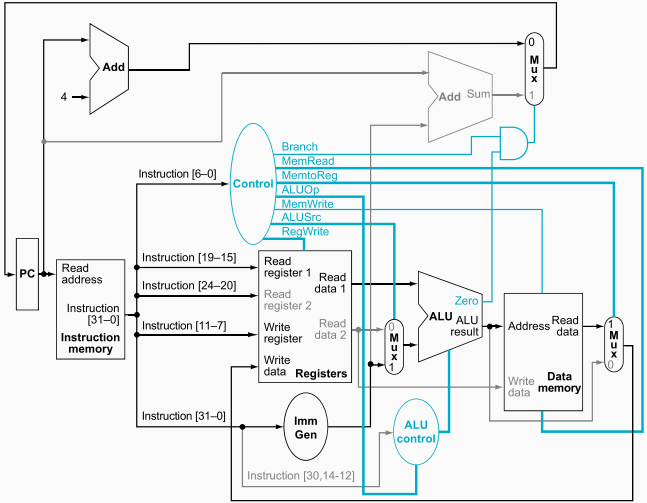
Full Datapath with Control



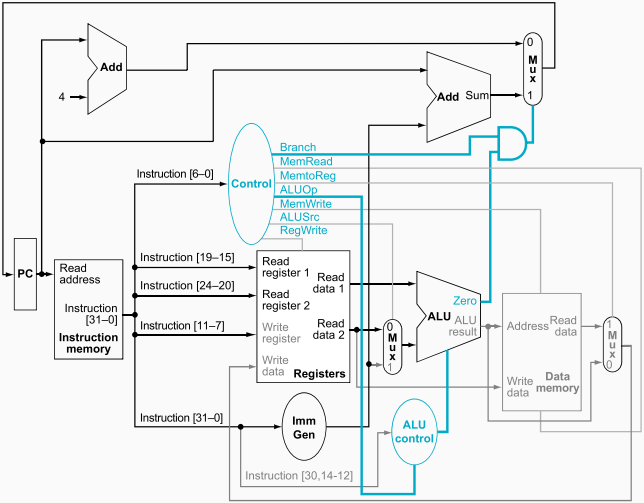
R-Type Instruction



Load Instruction



Branch-on-Equal Instruction



Finalizing Control

Input/Output	Signal Name	R-format	lw	sw	beq
Inputs	I[6]	0	0	0	1
	I[5]	1	0	1	1
	I[4]	1	0	0	0
	I[3]	0	0	0	0
	I[2]	0	0	0	0
	I[1]	1	1	1	1
	I[0]	1	1	1	1
Outputs	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

Performance Issues in a Single Cycle Implementation

- Longest delay determines clock period
  - Critical path: load instruction (5 elements)
  - Instruction memory → register file → ALU → data memory → register file

Performance Issues in a Single Cycle Implementation

- Longest delay determines clock period
  - Critical path: load instruction (5 elements)
  - Instruction memory → register file → ALU → data memory → register file
- Not feasible to vary period for different instructions

Performance Issues in a Single Cycle Implementation

- Longest delay determines clock period
  - Critical path: load instruction (5 elements)
  - Instruction memory → register file → ALU → data memory → register file
- Not feasible to vary period for different instructions
- Violates design principle: making the common case fast

Performance Issues in a Single Cycle Implementation

- Longest delay determines clock period
  - Critical path: load instruction (5 elements)
  - Instruction memory → register file → ALU → data memory → register file
- Not feasible to vary period for different instructions
- Violates design principle: making the common case fast
- We will improve performance by pipelining

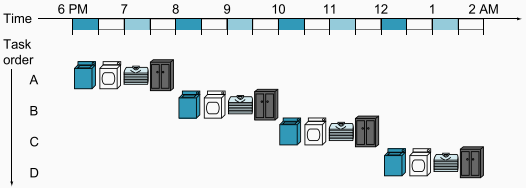
Pipelining Overview

Pipelining Analogy

- Pipelined laundry: overlapping execution
  - Parallelism improves performance

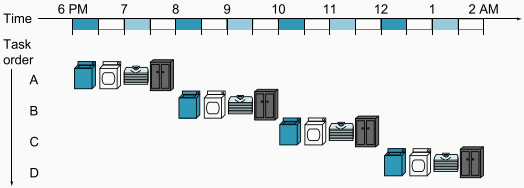
Pipelining Analogy

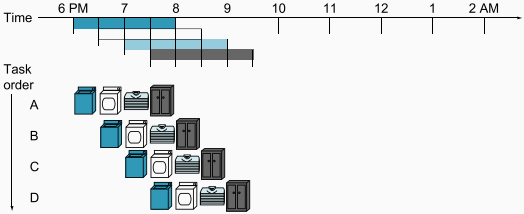
- Pipelined laundry: overlapping execution
  - Parallelism improves performance



Pipelining Analogy

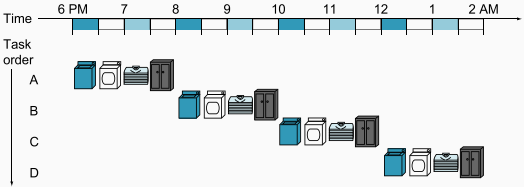
- Pipelined laundry: overlapping execution
  - Parallelism improves performance

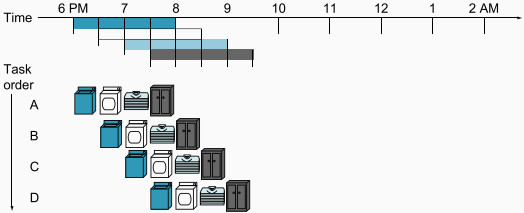




Pipelining Analogy

- Pipelined laundry: overlapping execution
  - Parallelism improves performance

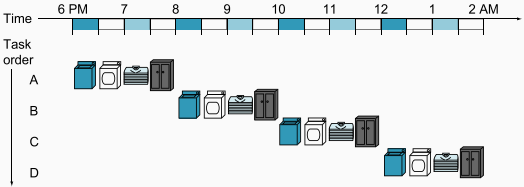


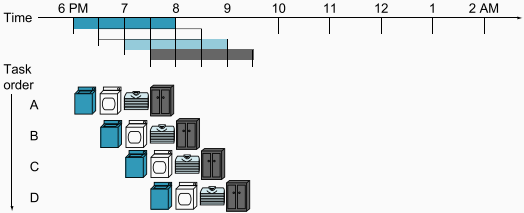


- For four loads, speedup $= {}^{8}/_{3.5} \approx 2.3$

Pipelining Analogy

- Pipelined laundry: overlapping execution
  - Parallelism improves performance





- For four loads, speedup $= {}^{8}/_{3.5} \approx 2.3$
- For non-stop, speedup $\approx 4 =$ number of stages

RISC-V Pipeline

RISC-V instructions classically take five steps, so five stages: one step per stage

RISC-V Pipeline

RISC-V instructions classically take five steps, so five stages: one step per stage

1. IF: Fetch instruction from memory

RISC-V Pipeline

RISC-V instructions classically take five steps, so five stages: one step per stage

1. IF: Fetch instruction from memory
2. ID: Read registers and decode the instruction

RISC-V Pipeline

RISC-V instructions classically take five steps, so five stages: one step per stage

1. IF: Fetch instruction from memory
2. ID: Read registers and decode the instruction
3. EX: Execute the operation or calculate an address

RISC-V Pipeline

RISC-V instructions classically take five steps, so five stages: one step per stage

1. IF: Fetch instruction from memory
2. ID: Read registers and decode the instruction
3. EX: Execute the operation or calculate an address
4. MEM: Access an operand in data memory (if necessary)

RISC-V Pipeline

RISC-V instructions classically take five steps, so five stages: one step per stage

1. IF: Fetch instruction from memory
2. ID: Read registers and decode the instruction
3. EX: Execute the operation or calculate an address
4. MEM: Access an operand in data memory (if necessary)
5. WB: Write the result back into a register (if necessary)

Pipeline Performance

- Assume times for stages are:
  - 100*ps* for register read or write
  - 200*ps* for other stages

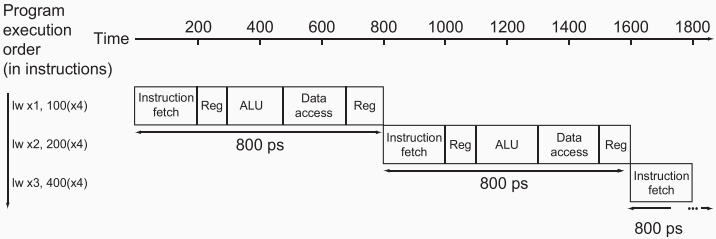
Pipeline Performance

- Assume times for stages are:
  - 100*ps* for register read or write
  - 200*ps* for other stages
- Compare pipelined datapath with single-cycle datapath

Instruction Class	Instruction Fetch	Register Read	ALU Operation	Data Access	Register Write	Total Time
Load word (`lw`)	200*ps*	100*ps*	200*ps*	200*ps*	100*ps*	800*ps*
Store word (`sw`)	200*ps*	100*ps*	200*ps*	200*ps*		700*ps*
R-format (`add`, `sub`, `and`, `or`)	200*ps*	100*ps*	200*ps*		100*ps*	600*ps*
Branch (`beq`)	200*ps*	100*ps*	200*ps*			500*ps*

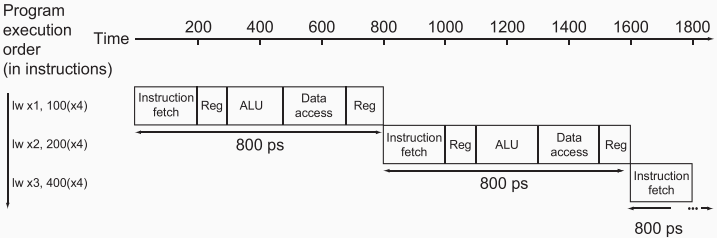
Single-Cycle vs Pipelined Execution

Single-cycle: $T_c = 800ps$

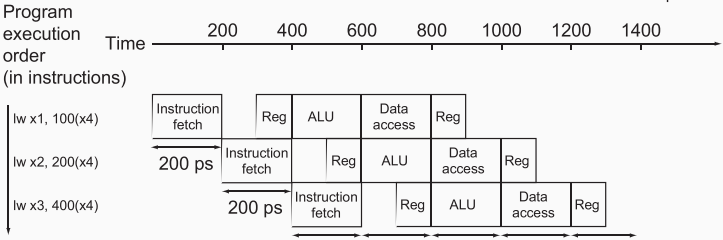


Single-Cycle vs Pipelined Execution

Single-cycle: $T_c = 800ps$



Pipelined: $T_c = 200ps$



Pipeline Speedup

- If all stages are balanced, i.e., take the same amount of time:

$$\text{Time between instructions}_{\text{pipelined}} = \frac{\text{Time between instructions}_{\text{non-pipelined}}}{\text{Number of stages}}$$

Pipeline Speedup

- If all stages are balanced, i.e., take the same amount of time:

$$\text{Time between instructions}_{\text{pipelined}} = \frac{\text{Time between instructions}_{\text{non-pipelined}}}{\text{Number of stages}}$$

- If not balanced, then the speedup is less

Pipeline Speedup

- If all stages are balanced, i.e., take the same amount of time:

$$\text{Time between instructions}_{\text{pipelined}} = \frac{\text{Time between instructions}_{\text{non-pipelined}}}{\text{Number of stages}}$$

- If not balanced, then the speedup is less
- Speedup is due to increased throughput, latency (time for each instruction) does not decrease

Pipelining and ISA Design

RISC-V ISA is designed for pipelining —

Pipelining and ISA Design

RISC-V ISA is designed for pipelining —

- All instructions are 32-bits
  - Easier to fetch and decode in one cycle
  - c.f., x86 has 1- to 17-byte instructions

Pipelining and ISA Design

RISC-V ISA is designed for pipelining —

- All instructions are 32-bits
  - Easier to fetch and decode in one cycle
  - c.f., x86 has 1- to 17-byte instructions
- Few and regular instruction formats
  - Can decode and read registers in one step

Pipelining and ISA Design

RISC-V ISA is designed for pipelining —

- All instructions are 32-bits
  - Easier to fetch and decode in one cycle
  - c.f., x86 has 1- to 17-byte instructions
- Few and regular instruction formats
  - Can decode and read registers in one step
- Load/store addressing
  - Can calculate address in the 3rd stage, access memory in the 4th stage

Pipeline Hazards

Pipeline hazards are situations that prevent starting the next instruction in the next cycle. There are three different types —

Pipeline Hazards

Pipeline hazards are situations that prevent starting the next instruction in the next cycle. There are three different types —

1. Structural Hazards: When a planned instruction cannot execute in the proper clock cycle because the hardware does not support the combination of instructions that are set to execute

Pipeline Hazards

Pipeline hazards are situations that prevent starting the next instruction in the next cycle. There are three different types —

1. Structural Hazards: When a planned instruction cannot execute in the proper clock cycle because the hardware does not support the combination of instructions that are set to execute
2. Data Hazards: When a planned instruction cannot execute in the proper clock cycle because data that are needed to execute the instruction are not yet available

Pipeline Hazards

Pipeline hazards are situations that prevent starting the next instruction in the next cycle. There are three different types —

1. Structural Hazards: When a planned instruction cannot execute in the proper clock cycle because the hardware does not support the combination of instructions that are set to execute
2. Data Hazards: When a planned instruction cannot execute in the proper clock cycle because data that are needed to execute the instruction are not yet available
3. Control Hazards: When deciding on the control actions depends on the previous instruction

Structural Hazards

- Conflict for the use of a resource

Structural Hazards

- Conflict for the use of a resource
- In a RISC-V pipeline with a single memory:
  - Load/store requires data access
  - Instruction fetch also requires the same and would have to stall for that cycle
  - Would cause a pipeline bubble

Structural Hazards

- Conflict for the use of a resource
- In a RISC-V pipeline with a single memory:
  - Load/store requires data access
  - Instruction fetch also requires the same and would have to stall for that cycle
  - Would cause a pipeline bubble
- Hence, pipelined datapaths require separate instruction/data memories, or, separate instruction/data caches

Data Hazards

An instruction depends on the completion of data access by a previous instruction

Data Hazards

An instruction depends on the completion of data access by a previous instruction

Example:

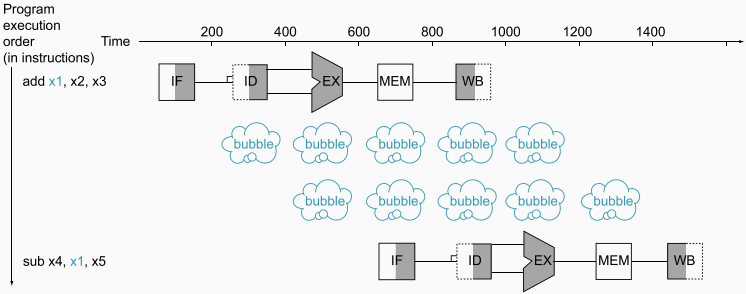
```
add x1, x2, x3
sub x4, x1, x5
```

Data Hazards

An instruction depends on the completion of data access by a previous instruction

Example:

```
add x1, x2, x3
sub x4, x1, x5
```



Forwarding (Bypassing)

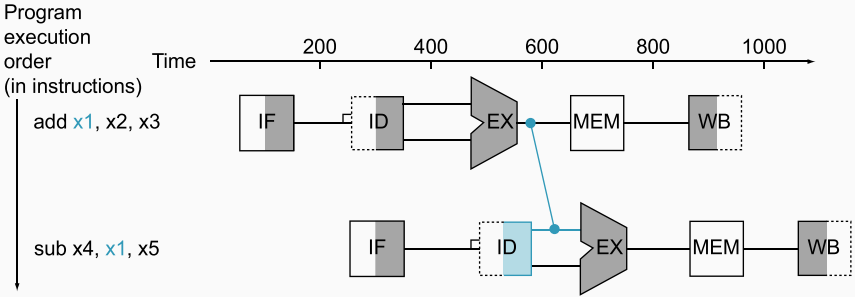
Use result when it is computer

- Do not wait for it to be stored in a register
- Requires extra connections in the datapath

Forwarding (Bypassing)

Use result when it is computer

- Do not wait for it to be stored in a register
- Requires extra connections in the datapath



Load-Use Data Hazard

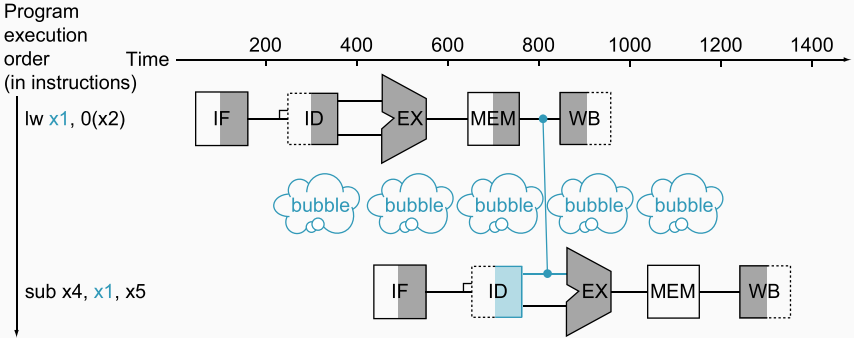
Cannot always avoid stalls by forwarding

- If value not computed when needed
- Cannot forward backward in time!

Load-Use Data Hazard

Cannot always avoid stalls by forwarding

- If value not computed when needed
- Cannot forward backward in time!



Code Scheduling to Avoid Stalls

Reorder code to avoid use of load result in the next instruction

Code Scheduling to Avoid Stalls

Reorder code to avoid use of load result in the next instruction

Example C Code (consider all variables in memory, addressable via x31):

```
a = b + e;
c = b + f;
```

Code Scheduling to Avoid Stalls

Reorder code to avoid use of load result in the next instruction

Example C Code (consider all variables in memory, addressable via x31):

```
a = b + e;
c = b + f;
```

```
lw x1, 0(x31) // Load b
lw x2, 4(x31) // Load e
add x3, x1, x2 // b + e
sw x3, 12(x31) // Store a
lw x4, 8(x31) // Load f
add x5, x1, x4 // b + f
sw x5, 16(x31) // Store c
```

Code Scheduling to Avoid Stalls

Reorder code to avoid use of load result in the next instruction

Example C Code (consider all variables in memory, addressable via x31):

```
a = b + e;
c = b + f;
```

```
lw x1, 0(x31) // Load b
lw x2, 4(x31) // Load e
add x3, x1, x2 // b + e [STALL]
sw x3, 12(x31) // Store a
lw x4, 8(x31) // Load f
add x5, x1, x4 // b + f [STALL]
sw x5, 16(x31) // Store c
```

Code Scheduling to Avoid Stalls

Reorder code to avoid use of load result in the next instruction

Example C Code (consider all variables in memory, addressable via x31):

```
a = b + e;
c = b + f;
```

```
lw x1, 0(x31) // Load b
lw x2, 4(x31) // Load e
add x3, x1, x2 // b + e [STALL]
sw x3, 12(x31) // Store a
lw x4, 8(x31) // Load f
add x5, x1, x4 // b + f [STALL]
sw x5, 16(x31) // Store c
```

```
lw x1, 0(x31)
lw x2, 4(x31)
lw x4, 8(x31) // Rearranged
add x3, x1, x2 // No stall
sw x3, 12(x31)
add x5, x1, x4 // No stall
sw x5, 16(x31)
```

Control Hazards

- Branch determines flow of control

Control Hazards

- Branch determines flow of control
  - Fetching next instruction depends on branch outcome

Control Hazards

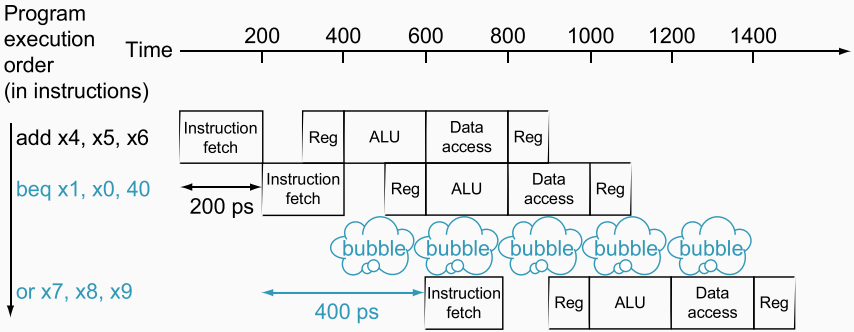
- Branch determines flow of control
  - Fetching next instruction depends on branch outcome
  - Pipeline cannot always fetch the correct instruction — still working on ID stage of branch

Control Hazards

- Branch determines flow of control
  - Fetching next instruction depends on branch outcome
  - Pipeline cannot always fetch the correct instruction — still working on ID stage of branch
- In RISC-V pipeline:
  - Need to compare registers and compute target early in the pipeline
  - Add hardware to do it in the ID stage

Stall on Branch

Wait until branch outcome is determined before fetching the next instruction



Branch Prediction

- Longer pipelines cannot readily determine branch outcome early: stall penalty becomes unacceptable

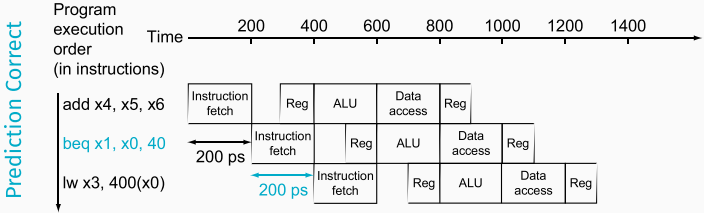
Branch Prediction

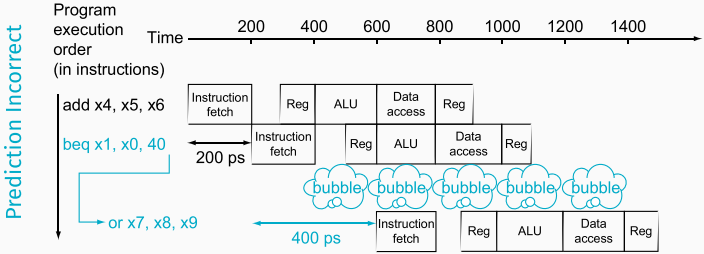
- Longer pipelines cannot readily determine branch outcome early: stall penalty becomes unacceptable
- Solution: Predict outcome of the branch: only stall when prediction is wrong

Branch Prediction

- Longer pipelines cannot readily determine branch outcome early: stall penalty becomes unacceptable
- Solution: Predict outcome of the branch: only stall when prediction is wrong
- In RISC-V pipeline:
  - Can predict branches not taken
  - Fetch instruction after branch, with no delay

Penalty of Incorrect Prediction





More Realistic Branch Prediction

- Static Branch Prediction
  - Based on typical branch behavior
  - Example: loop and if-statement branches
    - Predict backward branches taken
    - Predict forward branches not taken

More Realistic Branch Prediction

- Static Branch Prediction
  - Based on typical branch behavior
  - Example: loop and if-statement branches
    - Predict backward branches taken
    - Predict forward branches not taken
- Dynamic Branch Prediction
  - Hardware measures actual branch behavior, e.g., record recent history of each branch
  - Assume future behavior will continue the trend: when wrong, stall while re-fetching and update history

Pipeline Summary

The BIG Picture

- Pipeline improves performance by increasing instruction throughput
  - Executes multiple instructions in parallel
  - Each instruction has the same latency

Pipeline Summary

The BIG Picture

- Pipeline improves performance by increasing instruction throughput
  - Executes multiple instructions in parallel
  - Each instruction has the same latency
- Subject to hazards: structural, data, and control

Pipeline Summary

The BIG Picture

- Pipeline improves performance by increasing instruction throughput
  - Executes multiple instructions in parallel
  - Each instruction has the same latency
- Subject to hazards: structural, data, and control
- Instruction set design affects the complexity of pipeline implementation

To be continued in *The Processor: Part II*!